The Most Expensive Line of Code in Your Test Suite.

❌ THE ANTI-PATTERN

```
await page.waitForTimeout(5000);
```

Why arbitrary sleep timers cost scaling engineering teams **$35,000/year** in wasted CI compute, delayed pull requests, and broken developer trust.

AV

Alberto E. Vilano Reyna

Senior SDET & AI Quality Engineer • avilano.dev

Swipe →

The CI Flake Feedback Loop

Flaky tests aren't random. They follow a predictable failure cycle:

1. **Local Run Passes 100%**
   Your MacBook M3 has plenty of CPU headroom and an unthrottled browser context.

2. **GitHub Actions Runner Throttles**
   Under shared multi-tenant runner CPU contention, 5,000ms is not enough. The test flakes.

3. **Someone Bumps it to 10,000ms**
   Instead of fixing the DOM race condition, developers inflate timeouts across 150 specs.

4. **Pipeline Time Balloons to 35+ Min**
   Developers stop waiting for CI. They hit "Re-run all jobs" 3 times or bypass checks.

AV

Alberto E. Vilano Reyna
avilano.dev

Never Wait for Time.
Wait for State.

Replace arbitrary timer sleeps with Playwright's auto-waiting Web-First assertions.

❌ BRITTLE SLEEP (RACE CONDITION PRONE)

```
// Waits 5s regardless of whether network responded
await page.waitForTimeout(5000);
const visible = await page.locator('.success').isVisible();
expect(visible).toBeTruthy();
```

✅ AUTO-WAITING WEB-FIRST ASSERTION

```
// Passes the instant the DOM enters valid state
await page.getByRole('button', { name: 'Confirm Payment' }).click();
await expect(page.getByRole('alert')).toHaveText(/payment confirmed/i);
```

AV

Alberto E. Vilano Reyna

avilano.dev

Stop Logging in via UI on Every Single Test

Testing the login page 200 times per build is a colossal waste of pipeline runtime.

LEGACY APPROACH

26.6 min

200 specs × 8s UI login flows (filling inputs, clicking submit, waiting for redirect)

PLAYWRIGHT STORAGESTATE

< 50 ms

Authenticate once via API setup project, serialize cookies/JWT, inject directly into context

PLAYWRIGHT.CONFIG.TS

```
use: {
  // Every test starts authenticated instantly
  storageState: 'playwright/.auth/user.json',
}
```

AV

Alberto E. Vilano Reyna

avilano.dev

Ditch Fragile XPaths for Accessible ARIA Roles

CSS classes change with every Tailwind refactor. Accessible user roles never do.

❌ FRAGILE DOM SELECTORS (BREAKS ON CSS CHANGES)

```
page.locator('div.btn-container > button.sc-1234.btn-primary')
```

✅ SEMANTIC ACCESSIBLE ROLE (WCAG ALIGNED)

```
page.getByRole('button', { name: 'Submit Application' })
```

💡 The Double Win for Your Team

If Playwright can't find your button by its accessible name, screen reader users can't find it either. You get automated WCAG accessibility auditing for free.

AV

Alberto E. Vilano Reyna

avilano.dev

Ready to Fix Your Pipeline Once and for All?

4 architectural pillars to eliminate test flakiness permanently:

- ✓ **Auto-waiting web-first assertions** (zero sleep timers)
- ✓ **API storageState caching** (<50ms authenticated specs)
- ✓ **Accessible ARIA role locators** (resilient to CSS refactors)
- ✓ **Isolated test fixtures & CI sharding** (zero test cross-talk)

Need this architecture built in your codebase?

Book a 48-Hour Diagnostic Audit ($950, 100% credited) or partner on a 21-Day Sprint.

AV

Alberto E. Vilano Reyna

Senior SDET & AI Quality Engineer • avilano.dev

avilano.dev ↗